

Эмулятор оболочки - вариант 16

Практическая работа № 1 по конфигурационному управлению. Реализован этап 1 (REPL): графическое окно на Python и tkinter. REPL - цикл «ввод команды → выполнение → вывод → следующий ввод».

Запуск

Нужен Python 3.10 или новее с поддержкой tkinter (Tcl/Tk). Сторонние пакеты Python не используются; установка через pip не нужна.

Из корня репозитория:

```
./run.sh
```

Или:

```
python3 -B -m src
```

На Windows можно использовать `py -B -m src` из корня репозитория. Отдельная сборка не требуется. Скрипт `run.sh` работает и при вызове из другой рабочей директории.

Переменная `PYTHON` задаёт интерпретатор:

```
PYTHON=/opt/homebrew/bin/python3 ./run.sh
```

Если импорт `tinker` не работает, установите поддержку Tcl/Tk для используемого интерпретатора. Проверить её можно командой `python3 -m tkinter` - должно открыться тестовое окно.

Возможности и настройки

- Заголовок окна содержит имя VFS: `MyVFS`. На этапе 1 имя задаётся константой `VFS_NAME` в `src/gui.py`.
- Команда выполняется по Enter или кнопке «Выполнить».
- История содержит введённые команды, результаты и ошибки. Её можно прокручивать и копировать; редактирование отключено.
- Парсер поддерживает одинарные и двойные кавычки, экранирование, несколько пробелов и пустые аргументы в кавычках.
- Пустой ввод игнорируется. Решётка не начинает комментарий.
- Неизвестные команды, незакрытые кавычки, оборванное экранирование и неверное число аргументов выводят ошибку. После неё ввод продолжается.

Команда	Поведение на этапе 1
<code>ls [пути...]</code>	Печатает имя команды и список аргументов
<code>cd [путь]</code>	Печатает имя и аргументы; максимум один аргумент
<code>exit</code>	Закрывает окно; аргументы запрещены

`ls` и `cd` - заглушки: они не читают файлы и не меняют текущую папку. Их аргументы показываются как JSON-список, чтобы было видно границы строк. Загрузка ZIP, параметры командной строки и стартовые скрипты появятся на следующих этапах. Реальные команды ОС не запускаются.

Примеры для защиты

Введите команды по очереди в окне:

```
ls
ls documents "Мои документы"
cd "Мои документы"
cd 'Мои документы'
ls ""
abracadabra
cd one two
cd "незакрытая кавычка
exit now
ls
exit
```

Например, `cd "Мои документы"` выдаст:

Команда: `cd`

Аргументы: `["Мои документы"]`

Покажите имя VFS в заголовке и оба способа ввода: Enter и кнопку. Дополнительно нажмите Enter в пустом поле: результата быть не должно. Ошибочные команды должны оставить окно открытым, следующий `ls` должен выполняться. Последний `exit` закрывает окно.

Проверки

Из корня репозитория:

```
python3 -B -m unittest discover -s tests -v
```

Проверки не открывают GUI. Они проверяют кавычки, пустые аргументы, экранирование, заглушки, завершение и обработку ошибок. Окно проверяется вручную по сценарию выше.

Устройство проекта

- `src/shell.py`: `parse_command` разбирает строку; `execute_command` проверяет и выполняет команды. `CommandError` обозначает ошибку ввода; `CommandResult` содержит вывод и признак завершения.
- `src/gui.py`: `ShellWindow` создаёт окно, записывает историю и передаёт введённую строку обработчику; `main` запускает GUI.
- `src/__main__.py`: точка входа для `python3 -m src`.
- `tests/test_shell.py`: проверки обработчика команд.
- `run.sh`: скрипт запуска для macOS/Linux.

История этапов

Первый коммит содержит этап 1 (REPL). Последующие этапы сохраняются отдельными коммитами с описанием изменений.